Experiment Report

Course: Computer Programming

Experiment 1: C++ Basics & Expressions

Class:

Name:

Student ID:

Instructor:

Date: 2026.5.21

Experiment 1: Basics & Expressions

I. Objectives and Content

This experiment aims to practice C++ basics, which are the chapters 2-3 of the textbook.

The detailed goals of this experiment are as follows.

• Syntax Mastery: Understand preprocessor directives, main function structure, and namespace usage.

• Data Integrity: Distinguish between variables and constants, and understand memory allocation for different data types.

• Logic Development: Master operator precedence, associativity, and the side effects of assignment expressions.

• Precision Control: Learn to handle data type conversions (implicit and explicit) and format console output for readability.

Complete the following 8 programming tasks. Ensure all code includes proper documentation and follows the specific technical requirements for each problem.

II. Preparation

1. Install Visual Studio Code.
2. Install Compiler.
3. Open a folder and terminal in Visual Studio Code.
4. Set up the project folder.
5. Create .cpp files.
6. Compile and run.

III. Project Results

Task 1: Basic Structure & Documentation

1. **Description:** Detailed descriptions of the code.
   1) Use a Block Comment to display Author, Date, Description.

```
/*
 * Author: XXX
 * Date: 2026-05-21
 * Description: This program displays the user's name, department, and
student ID on three separate lines.
 */
```

   2) Output name line.

```
    // Output name line
    cout << "Name: XXX" << endl; // Displays the name
```

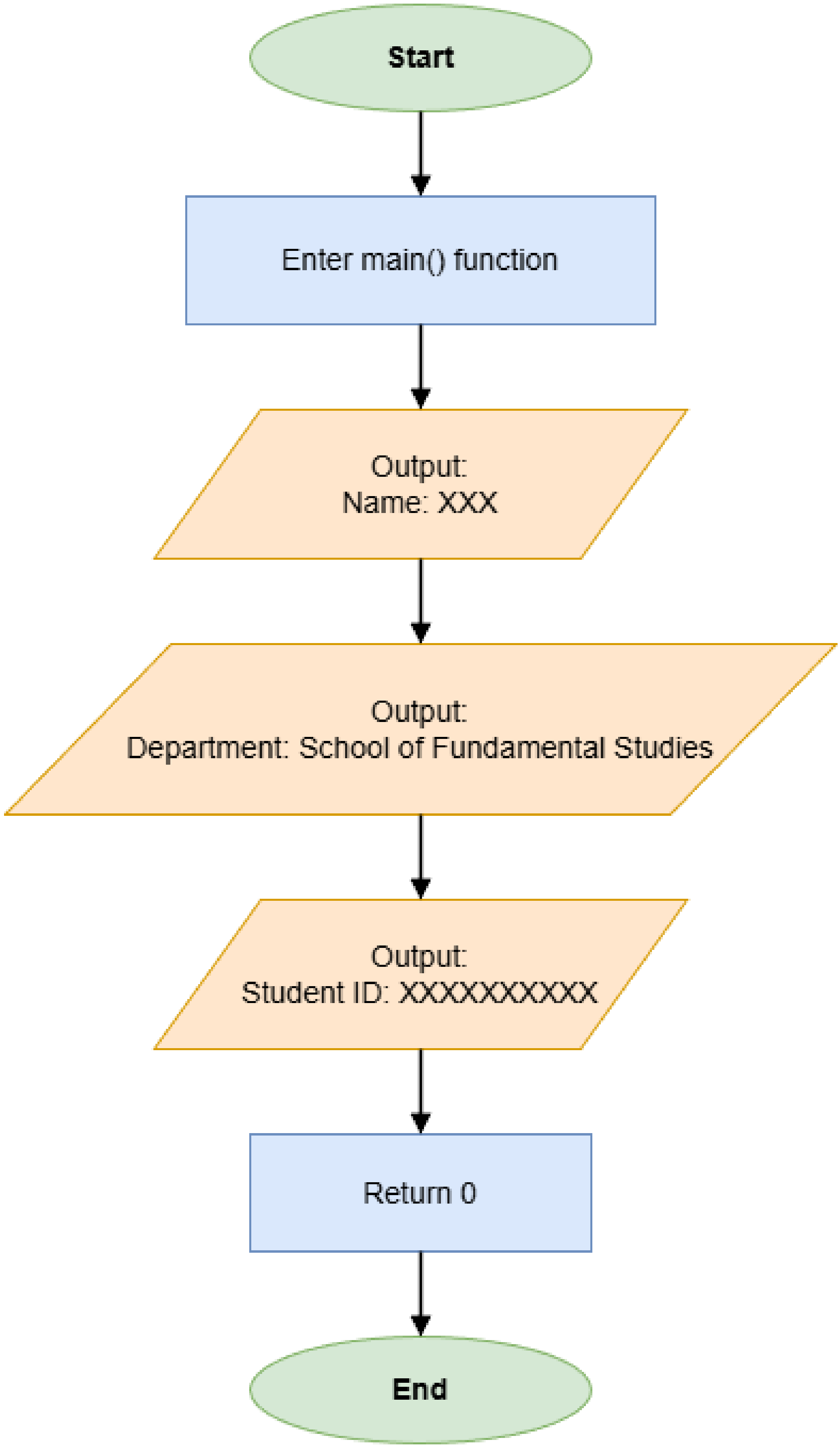
   3) Output department line.

```
    // Output department line
    cout << "Department: School of Fundamental Studies" << endl; //
Displays the department
```

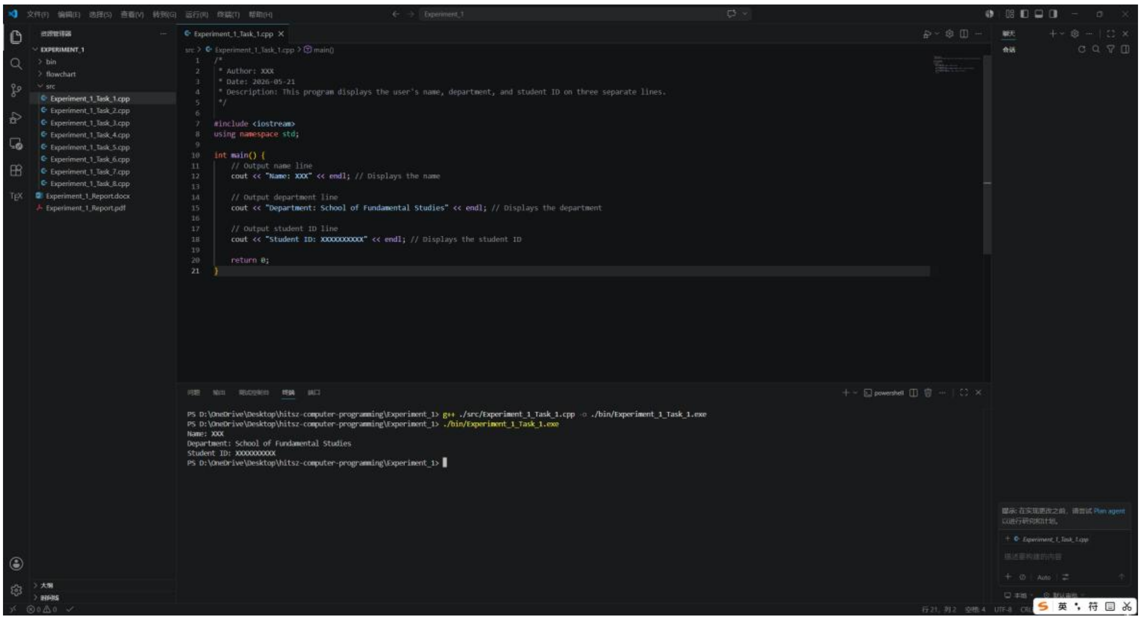
   4) Output student ID line.

```
    // Output student ID line
    cout << "Student ID: XXXXXXXXXX" << endl; // Displays the student ID
```

2. **Flow Diagram:**



3. **Results (Output Screenshot):** The screenshot of the output of the code.



Task 2: Coin Value Calculator

1. **Description:**
    1) Define each coin's value as a const unsigned int.

```
    const unsigned int quarter = 25;
    const unsigned int dime = 10;
    const unsigned int nickel = 5;
    const unsigned int penny = 1;
```

    2) Use unsigned int for coin counts and unsigned long for the totalValue.

```
    unsigned int numQuarters, numDimes, numNickels, numPennies;
    unsigned long totalValue = 0;
```

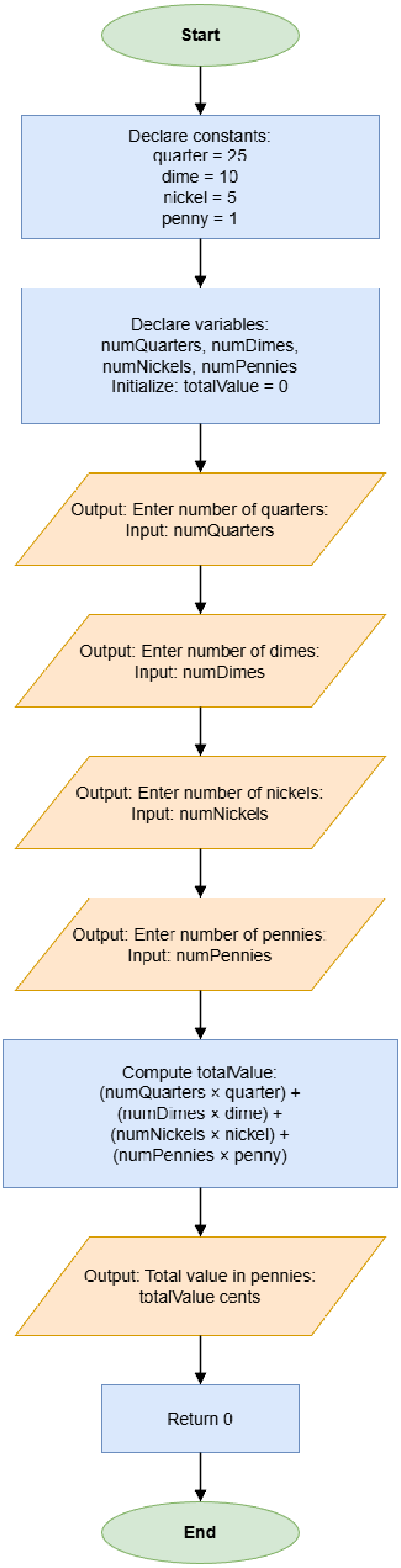
    3) Input the number of quarters, dimes, nickels and pennies.

```
    cout << "Enter number of quarters: ";
    cin >> numQuarters;
    cout << "Enter number of dimes: ";
    cin >> numDimes;
    cout << "Enter number of nickels: ";
    cin >> numNickels;
    cout << "Enter number of pennies: ";
    cin >> numPennies;
```

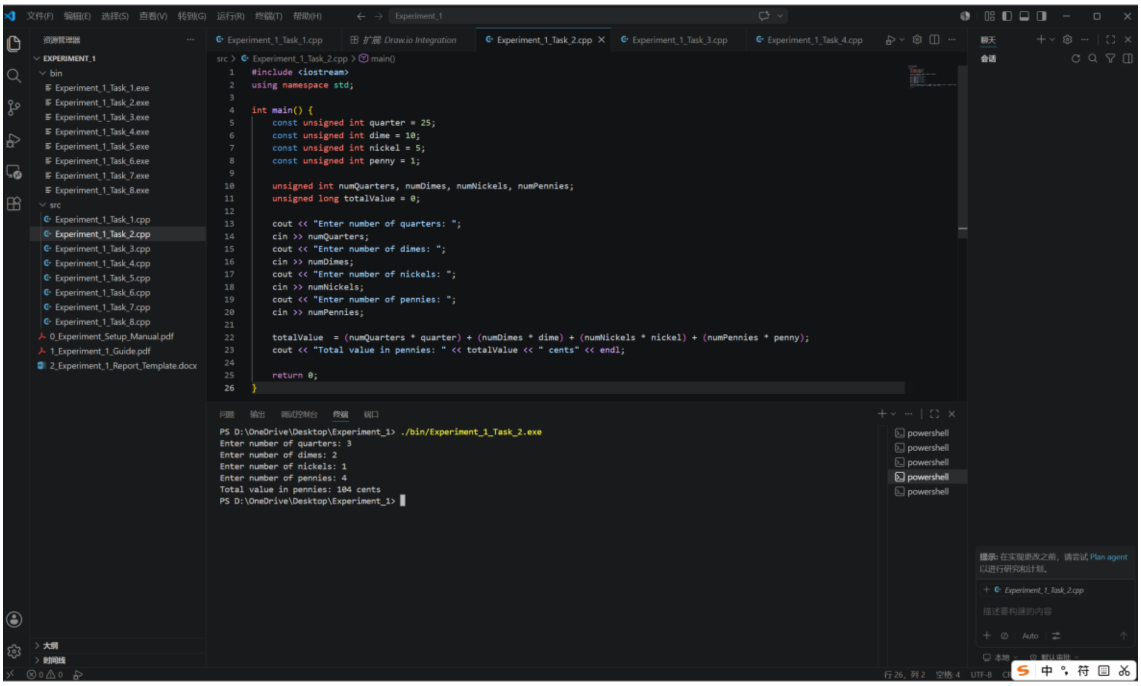
    4) Calculate totalValue and print it.

```
    totalValue  = (numQuarters * quarter) + (numDimes * dime) +
(numNickels * nickel) + (numPennies * penny);
    cout << "Total value in pennies: " << totalValue << " cents" << endl;
```

2. **Flow Diagram:**



3. **Results (Output Screenshot):**

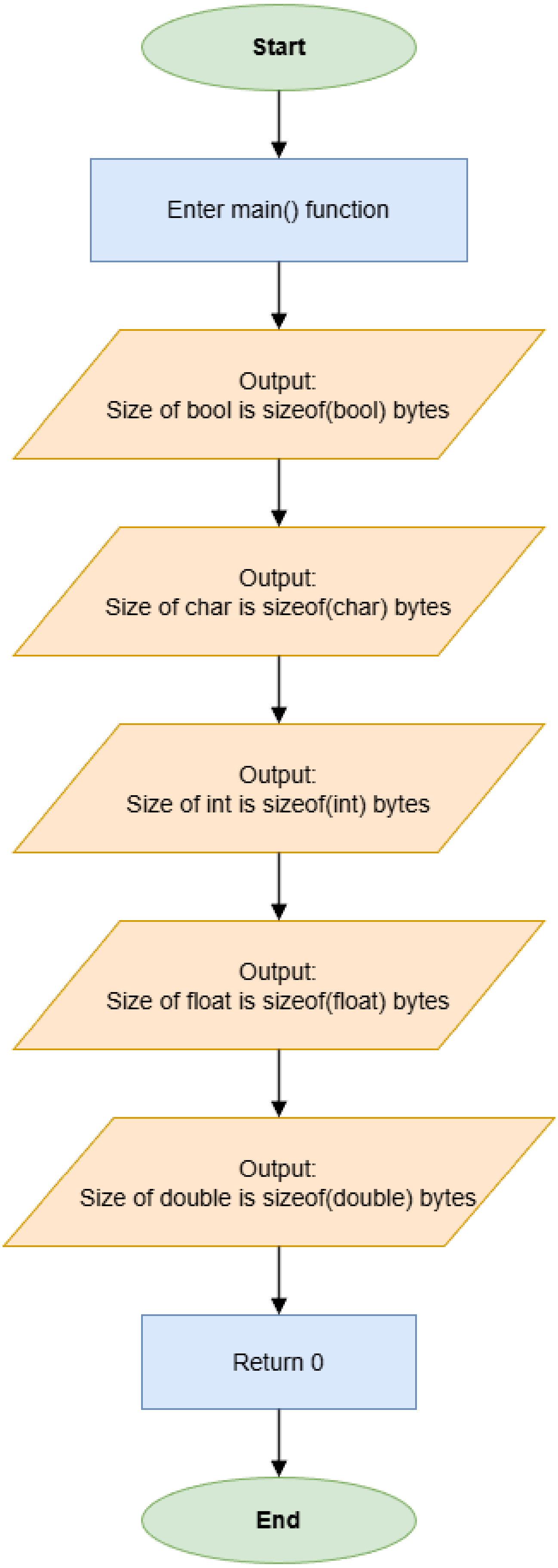


Task 3: Memory Size Analysis

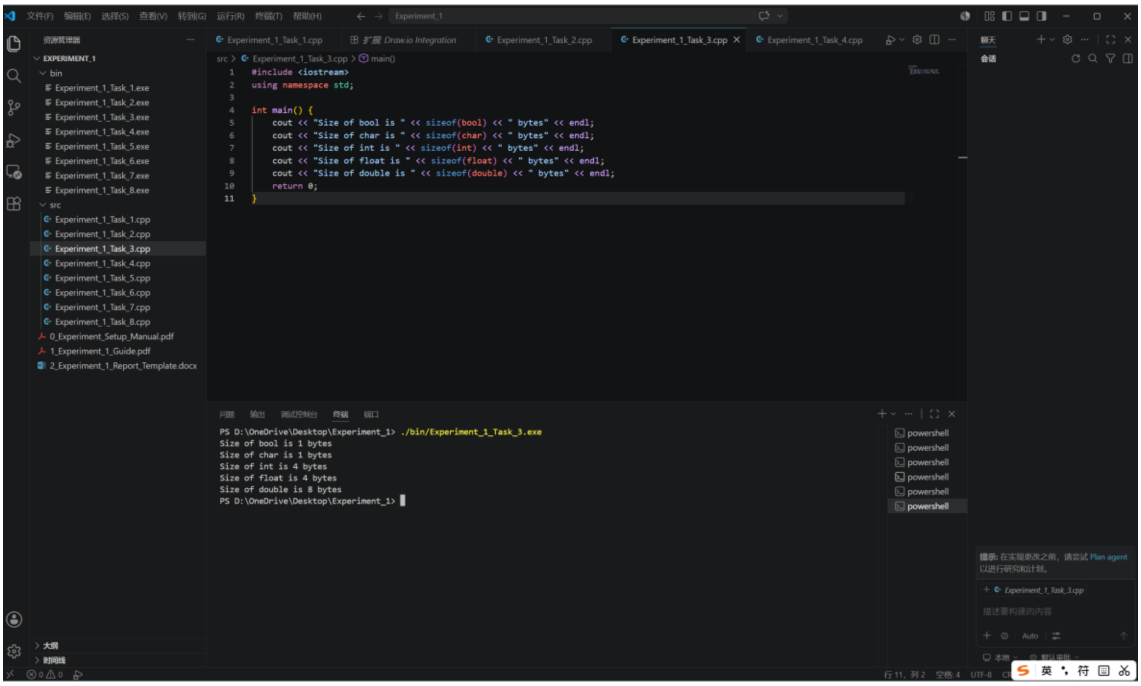
1. **Description:**
   1) Use the sizeof operator to print the memory size of different data types.

```
    cout << "Size of bool is " << sizeof(bool) << " bytes" << endl;
    cout << "Size of char is " << sizeof(char) << " bytes" << endl;
    cout << "Size of int is " << sizeof(int) << " bytes" << endl;
    cout << "Size of float is " << sizeof(float) << " bytes" << endl;
    cout << "Size of double is " << sizeof(double) << " bytes" << endl;
```

2. **Flow Diagram:**



3. **Results (Output Screenshot):**



Task 4: Character and ASCII Interaction

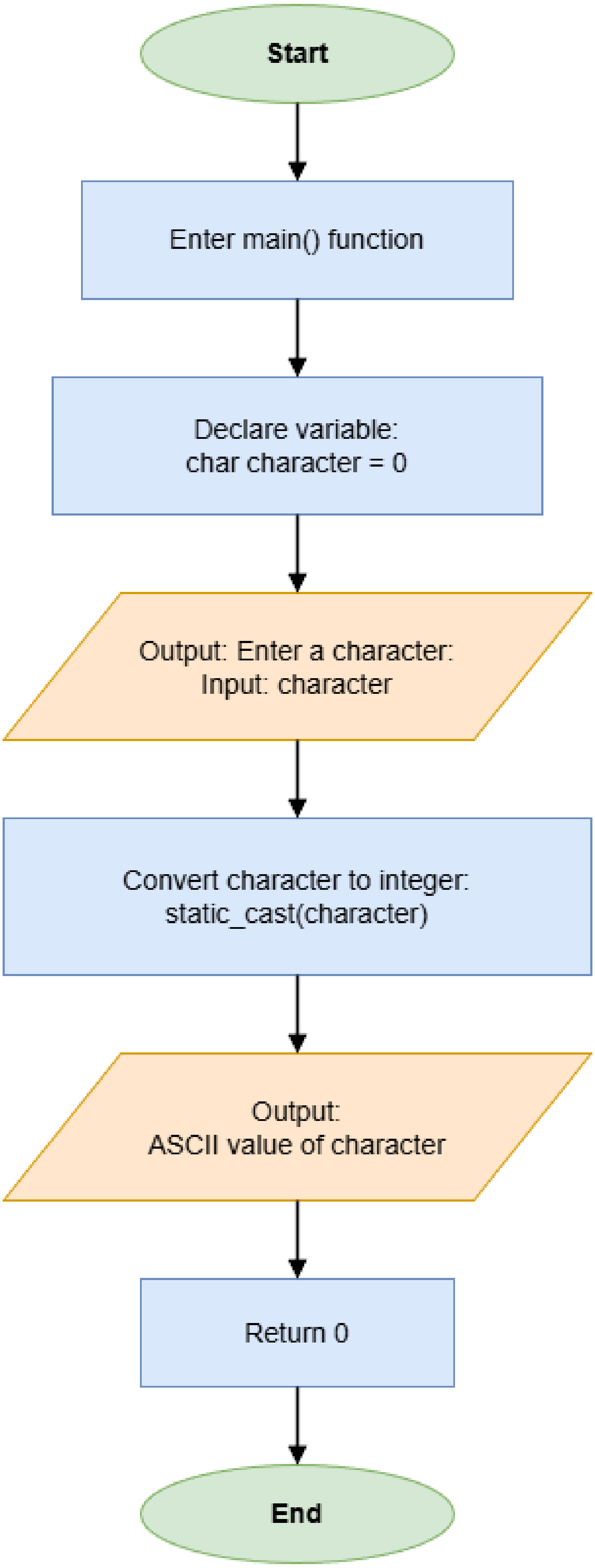
1. **Description:**
    1) Store the input in a char variable.

```
    char character = 0;

    cout << "Enter a character: ";
    cin >> character;
```

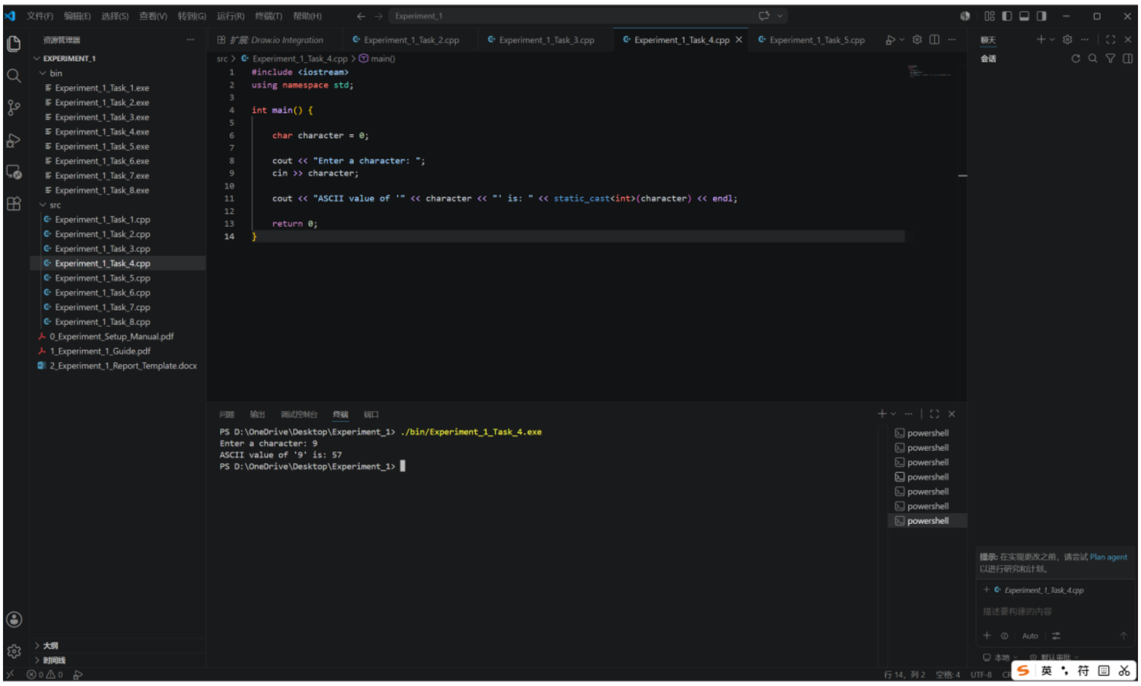
    2) Display that character's decimal ASCII value using static_cast<int>().

```
    cout << "ASCII value of '" << character << "' is: " <<
static_cast<int>(character) << endl;
```

2. **Flow Diagram:**



3. **Results (Output Screenshot):**



Task 5: Integer Division and Remainder Logic

1. **Description:**
    1) Define the variables A and B.

```
int A = 0;
int B = 0;
```

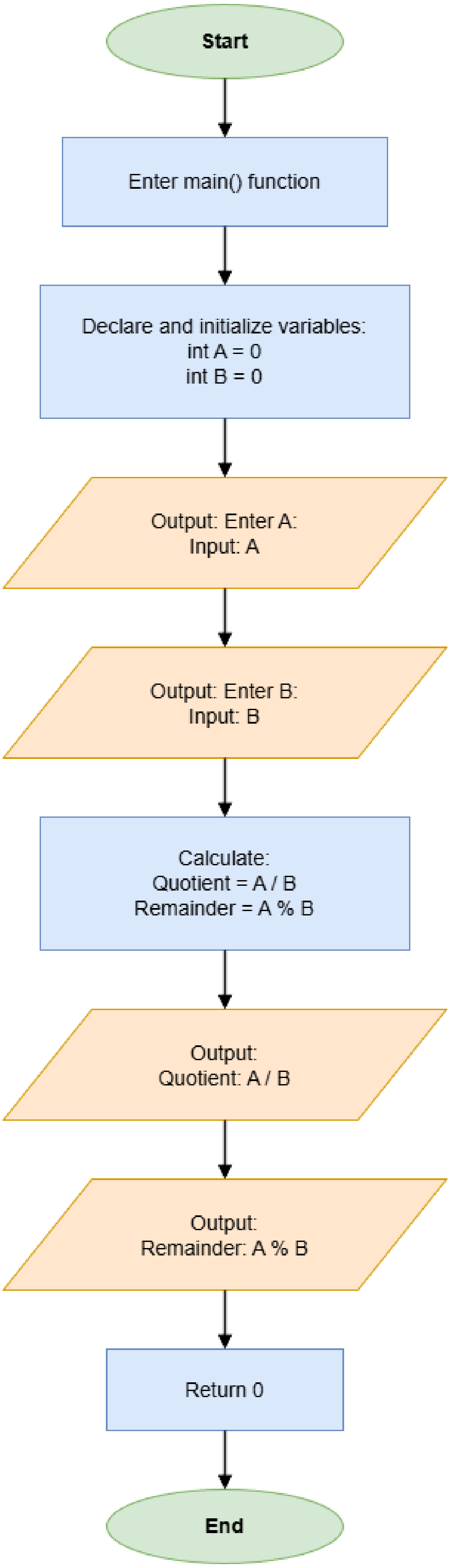
    2) Input A and B.

```
cout << "Enter A: ";
cin >> A;
cout << "Enter B: ";
cin >> B;
```

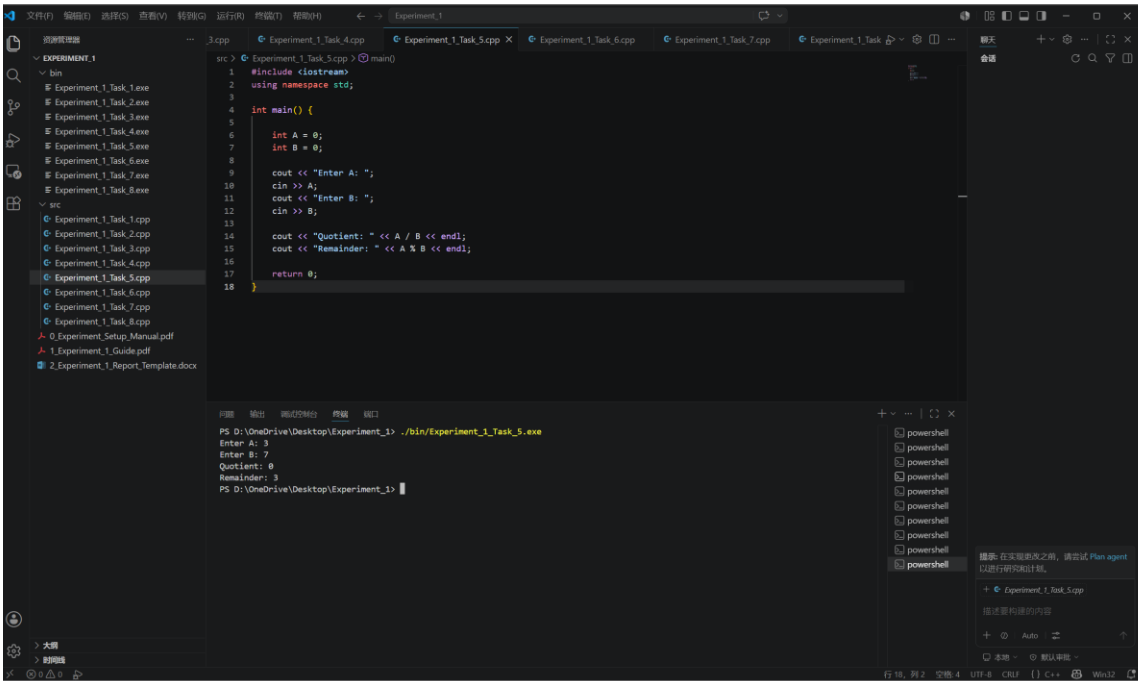
    3) Print the Quotient and Remainder.

```
cout << "Quotient: " << A / B << endl;
cout << "Remainder: " << A % B << endl;
```

2. **Flow Diagram:**



3. Results (Output Screenshot):



Task 6: Pythagorean Theorem

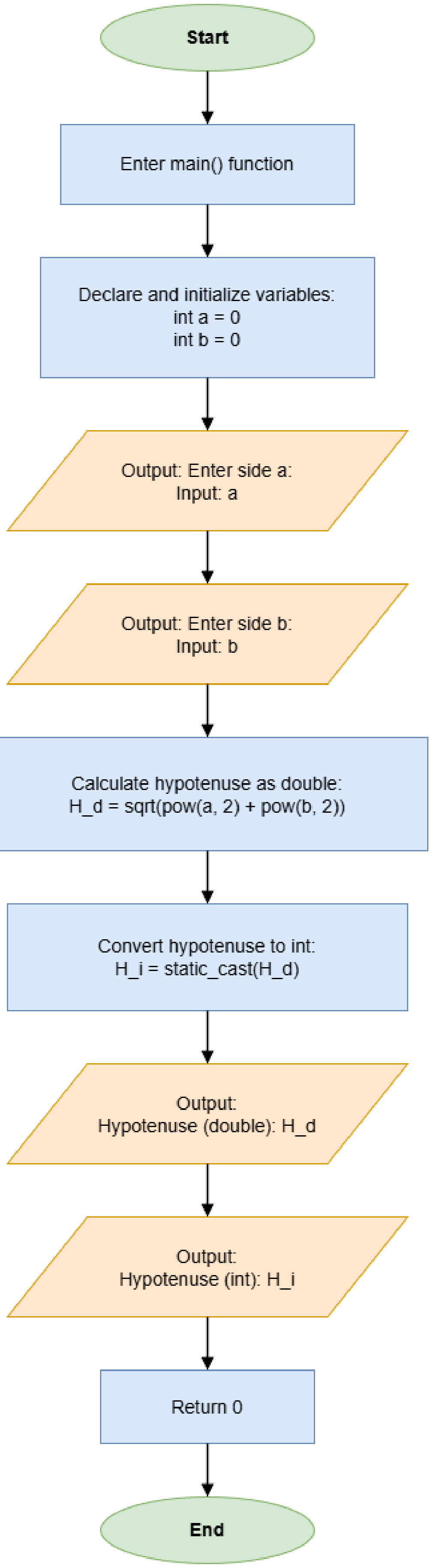
1. **Description:**
   1) Input two sides of a right triangle.

```
int a = 0;
int b = 0;

cout << "Enter side a: ";
cin >> a;
cout << "Enter side b: ";
cin >> b;
```

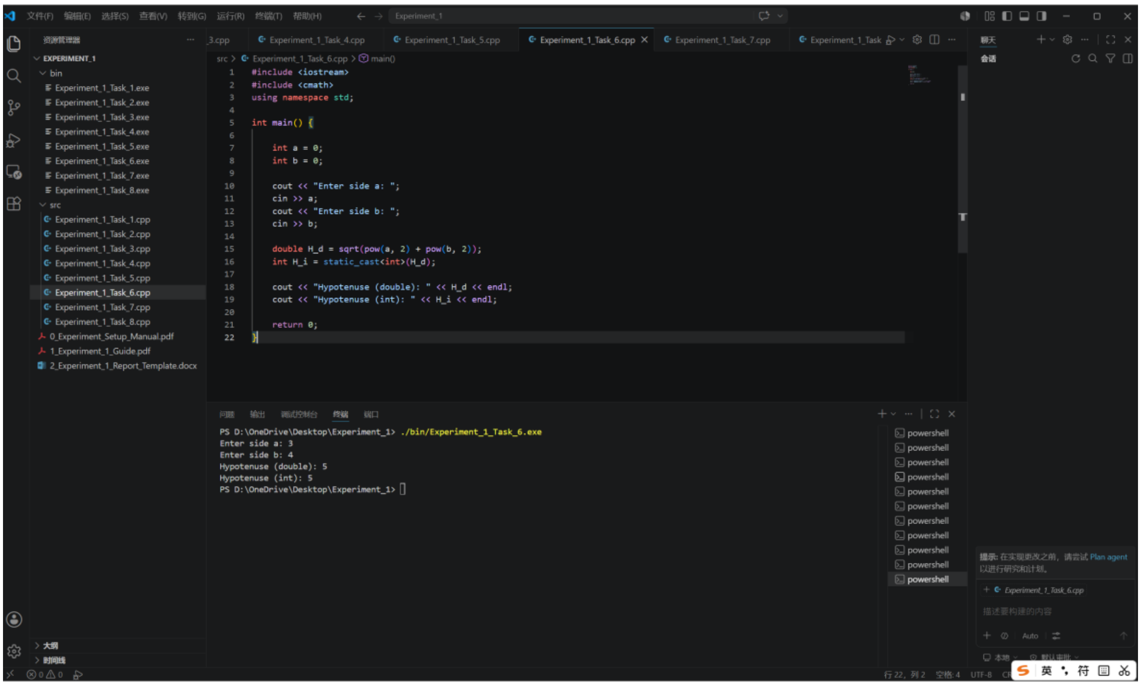
   2) Calculate the hypotenuse and print it in double and int.

```
double H_d = sqrt(pow(a, 2) + pow(b, 2));
int H_i = static_cast<int>(H_d);

cout << "Hypotenuse (double): " << H_d << endl;
cout << "Hypotenuse (int): " << H_i << endl;
```

2. **Flow Diagram:**



3. **Results (Output Screenshot):**



Task 7: Integer Overflow Observation

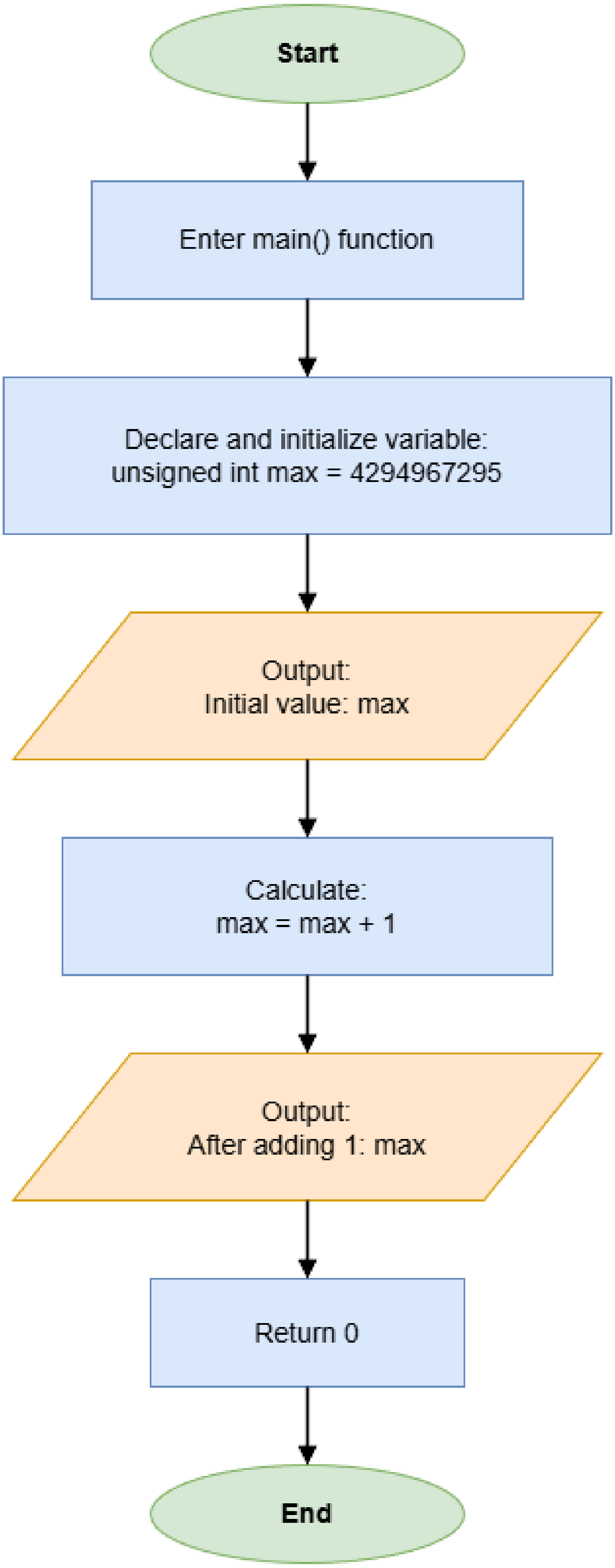
1. **Description:**
   1) Initialize an unsigned int with the value 4294967295 and display it.

```
unsigned int max = 4294967295;

cout << "Initial value: " << max << endl;
```

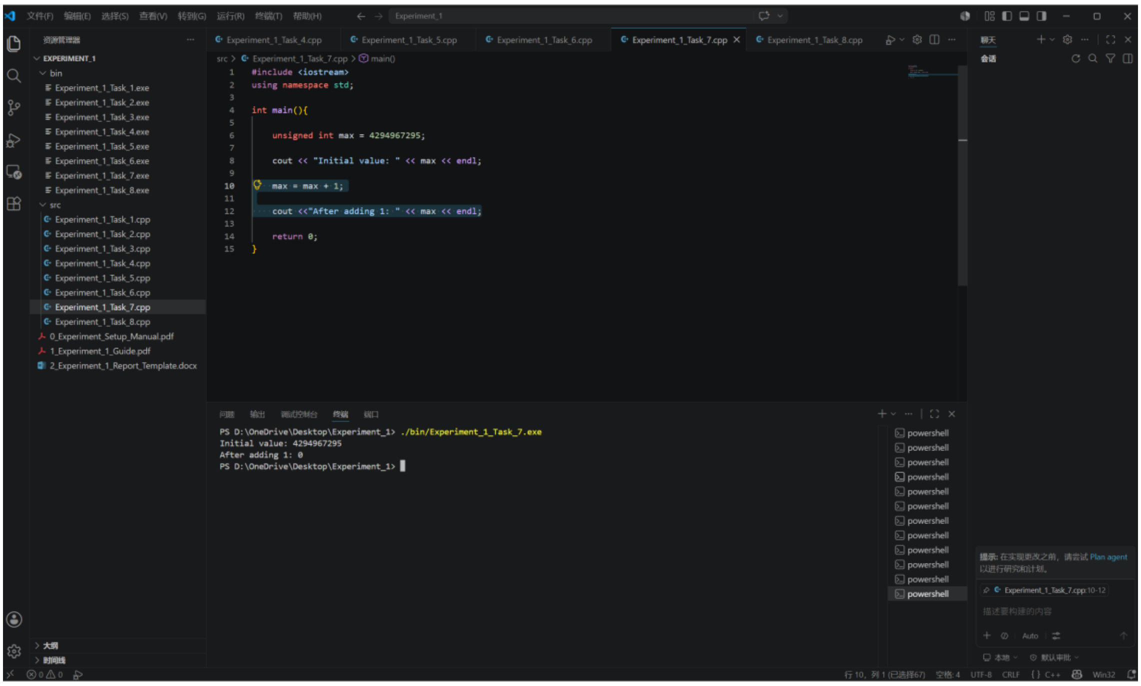
   2) Add 1 to it and print the result.

```
max = max + 1;

cout <<"After adding 1: " << max << endl;
```

2. **Flow Diagram:**



3. **Results (Output Screenshot):**



Task 8: Professional Invoice Formatting

1. **Description:**
   1) Input the prices of three items.

```
    double price1 = 0;
    double price2 = 0;
    double price3 = 0;

    cout << "Enter prices of three items: ";
    cin >> price1 >> price2 >> price3;
```

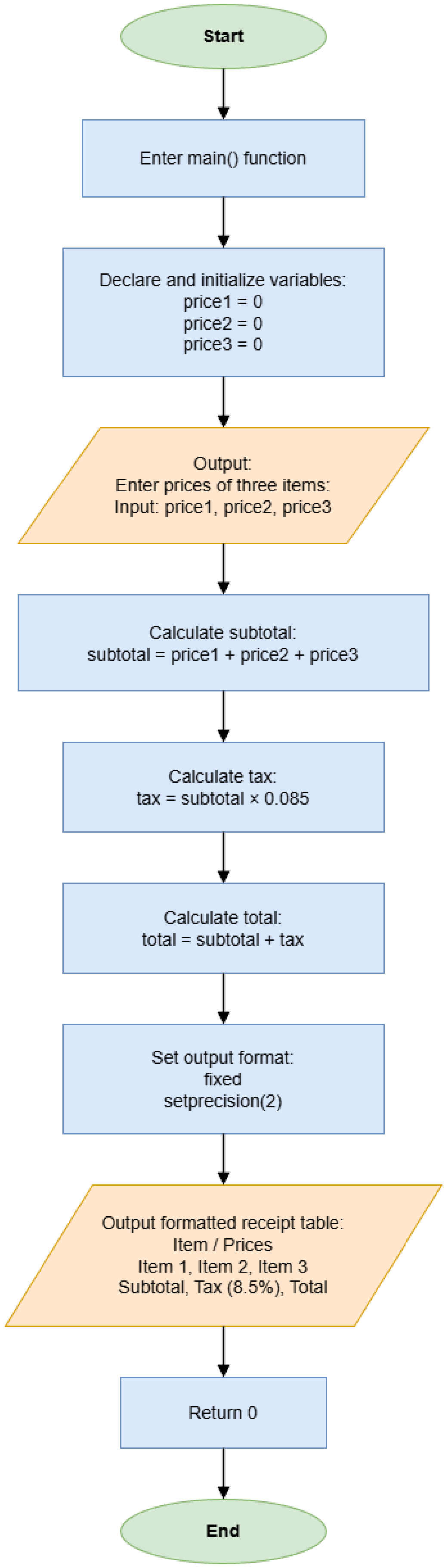
   2) Calculate subtotal, tax(8.5%), and total.

```
    double subtotal = price1 + price2 + price3;
    double tax = subtotal * 0.085;
    double total = subtotal + tax;
```

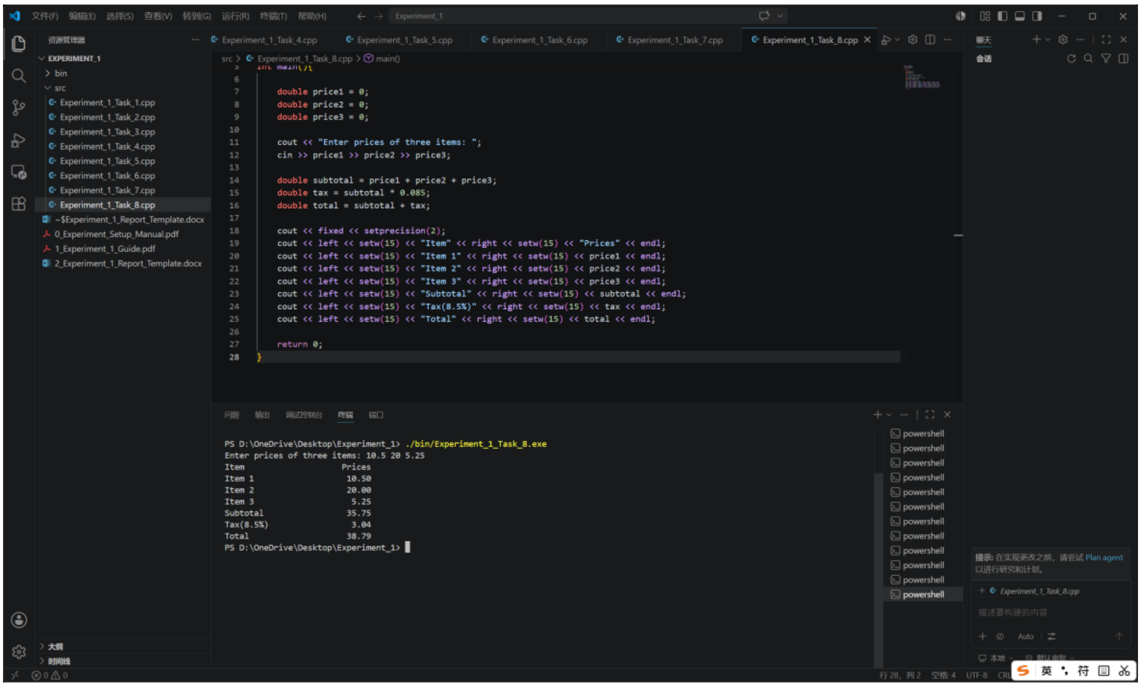
   3) Format console output using <iomanip>.

```
    cout << fixed << setprecision(2);
    cout << left << setw(15) << "Item" << right << setw(15) << "Prices"
<< endl;
    cout << left << setw(15) << "Item 1" << right << setw(15) << price1
<< endl;
    cout << left << setw(15) << "Item 2" << right << setw(15) << price2
<< endl;
    cout << left << setw(15) << "Item 3" << right << setw(15) << price3
<< endl;
    cout << left << setw(15) << "Subtotal" << right << setw(15) <<
subtotal << endl;
    cout << left << setw(15) << "Tax(8.5%)" << right << setw(15) << tax
<< endl;
    cout << left << setw(15) << "Total" << right << setw(15) << total <<
endl;
```

2. **Flow Diagram:**



3. **Results (Output Screenshot):**



IV. Extended Questions

1. In Task 5, when A = 3 and B = 7, why is the quotient 0?

   Because A and B are both integers, C++ performs integer division. The mathematical result of 3 / 7 is approximately 0.4286, but integer division discards the fractional part instead of rounding it. Therefore, the quotient is 0 and the remainder is 3.

2. In Task 7, why does the value become 0 after adding 1 to 4294967295?

   The value 4294967295 is the maximum value of a 32-bit unsigned int. Adding 1 produces 4294967296 mathematically, but this value is outside the storage range. Unsigned integer arithmetic wraps around modulo 2^{32}, so the result returns to 0.

V. Summary and Reflections

In this experiment, I practiced the basic knowledge of C++ programming, including program structure, comments, variables, constants, data types, operators, type conversion, mathematical functions, and formatted output. Through the eight tasks, I became more familiar with writing, compiling, running, and checking simple C++ programs.

In Task 1, I learned the basic structure of a C++ program and practiced using comments to make the code clearer.

In Task 2, I used constants and unsigned integer variables to calculate the total value of coins, which helped me understand the difference between constants and variables.

In Task 3, I used the sizeof operator to observe the memory size of different data types.

In Task 4, I converted a character to its ASCII value using static_cast<int>(), which helped me understand character storage and type conversion.

In Task 5, I practiced integer division and the remainder operator. I learned that integer division discards the fractional part.

In Task 6, I used the Pythagorean theorem with sqrt() and pow() to calculate the hypotenuse, and observed the difference between double and int results.

In Task 7, I observed unsigned integer overflow, which showed that computer arithmetic is limited by data type ranges.

In Task 8, I used <iomanip> to format an invoice output, making the result clearer and more professional.

During the experiment, the main difficulties I encountered were how to compile .cpp files correctly and how to use <iomanip> to control decimal places and format the output professionally. At first, I was not familiar with the compiling process in Visual Studio Code, so I needed to learn how to open the terminal, use the correct command, and run the generated program. In Task 8, I also spent time learning how to use fixed, setprecision(), setw(), left, and right to make the invoice output neat and readable. By testing the programs several times and asking AI, I gradually solved these problems. Overall, this experiment helped me build a basic foundation in C++ programming and improved my ability to write and run simple programs.